

Month 3 详细指导书：LangGraph 多 Agent 原型、Human-in-the-loop、Case Memory 与高级产品展示

AI-first LQCD Meson DA Analysis Agent Training Plan

项目指导文档

Month 3 总目标

第三个月的目标是把前两个月已经建立好的 **config-driven CLI**、**Pydantic schema**、**pytest**、**Snakemake workflow**、**Analysis Engineer wrapper**、**Fit Strategy Post-doc**、**Critical Referee** 和 **rule-based Analysis Coordinator** 进一步升级为一个真正意义上的 **stateful multi-agent research prototype**。

学生需要完成一个可以展示的初级高级成品：系统能够从一个 **golden example** 或小规模真实分析 case 出发，尽可能自动完成数据检查、workflow 执行、fit 诊断、critical referee review、Analysis Coordinator 决策、targeted rerun 建议、case memory 更新和最终报告生成，并只在关键风险节点触发 human approval gate。

本月重点不是写一个炫酷界面，而是让系统具备现代科研 AI agent 的核心能力：状态可追踪、决策可解释、失败可诊断、默认自动化、关键点人工介入、经验可积累、流程可复现。

目录

1	Month 3 的定位：从 workflow prototype 到 agent prototype	4
1.1	Snakemake 和 LangGraph 的职责分工	4
2	Month 3 四周安排总览	5
3	Week 9：建立 LangGraph 多 Agent State Machine 原型	5
3.1	本周目标	5
3.2	学生需要学习什么	6
3.3	推荐目录结构	6
3.4	Step 1: 定义 AgentState	7
3.5	Step 2: 把已有模块封装成 graph nodes	7
3.6	Step 3: 定义 graph edge 逻辑	8
3.7	Step 4: 保存 graph state	9
3.8	Week 9 Codex Prompt	9

3.9	Week 9 命令示例	10
3.10	Week 9 常见错误	10
3.11	Week 9 导师检查问题	10
3.12	Week 9 验收标准	10
4	Week 10: Risk-Based Human Approval Gate 与 Case Memory	12
4.1	本周目标	12
4.2	学生需要理解的理念: Risk-Based Autonomy	12
4.3	Step 1: 定义 Risk-Based Autonomy 与 AI 决策边界	12
4.4	Step 2: 定义 Human Approval Packet 文件结构	13
4.5	Step 3: 实现条件分支 Graph (替代线性流程)	14
4.6	Step 4: 定义 Targeted Rerun Request (含风险上下文)	15
4.7	Step 5: 建立 Case Memory (按风险等级分层)	16
4.8	Week 10 命令示例	16
4.9	Week 10 常见错误	17
4.10	Week 10 导师检查问题	17
4.11	Week 10 Codex Prompt	17
4.12	Week 10 验收标准	18
5	Week 11: Evaluation Harness、Failure Cases 与自动报告生成	19
5.1	本周目标	19
5.2	Step 1: 建立 evaluation cases 目录	19
5.3	Step 2: 定义 expected behavior	19
5.4	Step 3: 实现 run_evaluation.py	20
5.5	Step 4: 自动生成 final analysis report	20
5.6	Step 5: 加入 git commit hash 和 environment info	22
5.7	Week 11 Codex Prompt	22
5.8	Week 11 命令示例	23
5.9	Week 11 常见错误	24
5.10	Week 11 导师检查问题	24
5.11	Week 11 验收标准	24
6	Week 12: 最终 Demo、文档整理与产品化设计	25
6.1	本周目标	25
6.2	Step 1: 整理一键 demo 命令	25
6.3	Step 2: 整理最终输出目录	26
6.4	Step 3: 补全 README	26
6.5	Step 4: 画 architecture diagram	27
6.6	Step 5: 准备最终汇报	27
6.7	Week 12 Codex Prompt	28
6.8	Week 12 命令示例	28

6.9	Week 12 常见错误	29
6.10	Week 12 导师检查问题	29
6.11	Week 12 验收标准	29
7	Month 3 最终交付清单	30
8	导师验收 checklist	30
8.1	Agent architecture	30
8.2	Risk-Based Human Approval Gate	31
8.3	Targeted rerun	31
8.4	Memory	31
8.5	Evaluation	31
8.6	科研可靠性	31
9	升华：理想产品应该呈现出的效果	32
9.1	理想中的产品不是“自动跑脚本”，而是“科研分析驾驶舱”	32
9.2	产品最终应该让导师看到什么	32
9.3	未来升级方向：如何变成更高级的系统	33
9.3.1	升级 1：从 rule-based Analysis Coordinator 到 LLM-assisted Analysis Coordinator	33
9.3.2	升级 2：从 YAML memory 到 SQLite / DuckDB analysis memory	33
9.3.3	升级 3：加入 richer observability	34
9.3.4	升级 4：从单 case 到 ensemble-level campaign	34
9.3.5	升级 5：引入 multi-agent debate，但必须有风险控制	34
9.3.6	升级 6：加入真实 LQCD physics checks	35
9.4	如何变成“the best”	35
10	给学生的最终提醒	36

1 Month 3 的定位：从 workflow prototype 到 agent prototype

前两个月已经解决了两个基础问题：

1. **Month 1**: 把 golden example 复现出来，并逐步整理成 config-driven Python CLI、schema、pytest、manifest、result 和 log。
2. **Month 2**: 把 CLI 放入 Snakemake workflow，并加入 Analysis Engineer wrapper、Fit Strategy Postdoc、Critical Referee、rule-based Analysis Coordinator 和 decision log。

第三个月要解决的问题是：

如何让这些已经写好的模块不只是彼此独立运行，而是组成一个有状态、有分工、有回退、有审批、有记忆的科研 agent 系统？

因此 Month 3 的核心是：

- 引入 **LangGraph** 或等价 **state-machine** 机制管理多 agent 的状态传递；
- 把 Correlator Analyst、Analysis Engineer、Fit Strategy Postdoc、Critical Referee、Analysis Coordinator、Human Approval Gate、Analysis Memory Curator 串成一个清晰流程；
- 支持 **targeted rerun**，也就是 Critical Referee 或 Analysis Coordinator 发现问题后，不是全量重跑，而是精确要求 Analysis Engineer 重跑某一步；
- 加入 **human approval gate**，高风险结果不能自动进入 accepted memory；
- 建立 **case memory**，保存 accepted / rejected / suspicious fit cases；
- 建立 **evaluation harness**，用固定 test cases 检查 agent 决策是否稳定；
- 生成 **final demo report**，让导师能够一眼看出系统的价值、边界和下一步升级方向。

1.1 Snakemake 和 LangGraph 的职责分工

第三个月必须明确一个原则：**Snakemake** 和 **LangGraph** 不是互相替代的工具，而是处理不同层面的 DAG。

工具	主要职责	在本项目中的作用
Snakemake	文件依赖 DAG、增量重跑、批量计算、HPC workflow	管理 audit、fit、plot、summary、diagnostics 等具体计算步骤
LangGraph	Agent 状态 DAG、决策流、human approval gate、失败回退、memory update	管理 Correlator Analyst、Analysis Engineer、Critical Referee、Analysis Coordinator、Human Approval、Analysis Memory Curator 的协作

Pydantic	输入输出 schema、类型检查、结构化协议	约束 config、task、result、decision、memory 的格式
pytest	可测试性、回归测试、失败案例固化	保证 Codex 生成代码和 agent 决策不会无声破坏系统
case memory	经验积累、accepted/rejected case 管理	帮助 Analysis Coordinator 从历史 case 中学习和保持一致性

学生需要理解：**Snakemake** 管 “算什么、什么时候重算”；**LangGraph** 管 “谁判断、判断后下一步做什么”。

2 Month 3 四周安排总览

周次	主题	主要交付
Week 9	LangGraph state machine 原型	定义 AgentState; 把 Correlator Analyst、Analysis Engineer、Fit Strategy Postdoc、Critical Referee、Analysis Coordinator 串成可运行 graph; 生成 graph run log
Week 10	Human approval、targeted rerun 与 case memory	实现 approval gate、rerun request、memory update; 区分 accepted/rejected/suspicious case
Week 11	Evaluation harness、failure cases 与报告自动生成	构造 good/ambiguous/bad 三类固定 case; 测试 Analysis Coordinator 决策稳定性; 生成 final analysis report
Week 12	最终 demo、文档、产品化设计与未来升级路线	完成一键 demo; 整理 README、architecture diagram、advisor checklist、future roadmap; 准备最终汇报

3 Week 9: 建立 LangGraph 多 Agent State Machine 原型

3.1 本周目标

Week 9 的目标不是马上让 LLM 做复杂物理判断，而是先把已有模块用一个清楚的 state machine 串起来。

本周结束时，学生应该能够运行类似命令：

```
python -m lqcd_agent.graph.run_graph \  
--case configs/cases/l64c64a076_demo.yaml \  
--output-dir outputs/graph_demo/l64c64a076
```

系统应自动执行：

1. Correlator Analyst 检查输入数据和配置；

2. Analysis Engineer 调用 Snakemake 或 CLI 运行分析;
3. Fit Strategy Postdoc 读取 `summary.csv` 并生成 `fit_strategy_report.json`;
4. Matrix Element Analyst 读取 bare ME 输出并生成 `matrix_element_diagnostics.json`;
5. Fit Strategy Postdoc 最终确认 `summary.csv` 并生成 `fit_strategy_report.json`;
6. Critical Referee 生成 `critical_referee_report.json`;
7. Analysis Coordinator 生成 `decision_log.json`;
8. graph 保存完整状态到 `graph_state.json`。

3.2 学生需要学习什么

学生需要学习的不是 LangGraph 的所有高级功能，而是以下核心概念：

- 什么是 state;
- 什么是 node;
- 什么是 edge;
- 为什么每个 node 都应该读入 state、返回更新后的 state;
- 为什么 node 之间不能传递随意的自由文本，而要传递结构化字段;
- 如何把失败状态、警告、输出路径和 decision 都记录在 state 中。

3.3 推荐目录结构

在现有 repo 中增加：

```
src/lqcd_agent/  
  graph/  
    __init__.py  
    state.py  
    nodes.py  
    edges.py  
    run_graph.py  
    io.py  
  memory/  
    __init__.py  
    store.py  
    schema.py  
  
tests/  
  test_graph_state.py  
  test_graph_smoke.py
```

3.4 Step 1: 定义 AgentState

建议用 Pydantic 或 TypedDict 定义统一的 graph state。第一版可以简单，但字段必须清楚。

```
from pydantic import BaseModel, Field
from typing import Literal

class AgentState(BaseModel):
    case_id: str
    case_config_path: str
    output_dir: str

    status: Literal["initialized", "running", "success", "failed", "needs_human_approval"] = "initialized"
    current_stage: str = "start"

    correlator_diagnostics_path: str | None = None
    analysis_engineer_result_path: str | None = None
    summary_csv_path: str | None = None
    fit_strategy_report_path: str | None = None
    critical_referee_report_path: str | None = None
    decision_log_path: str | None = None
    final_report_path: str | None = None

    risk_level: Literal["low", "medium", "high", "unknown"] = "unknown"
    human_approval_required: bool = False
    human_approval_status: Literal["not_required", "pending", "approved", "rejected"] = "not_required"

    rerun_requested: bool = False
    rerun_reason: str | None = None
    rerun_target: str | None = None

    warnings: list[str] = Field(default_factory=list)
    errors: list[str] = Field(default_factory=list)
    event_log: list[str] = Field(default_factory=list)
```

3.5 Step 2: 把已有模块封装成 graph nodes

每个 node 应该是一个小函数，输入 state，输出更新后的 state。

```
def correlator_analyst_node(state: AgentState) -> AgentState:
    state.current_stage = "data_audit"
    # call existing Correlator Analyst
    # save audit_report.json
```

```

state.correlator_diagnostics_path = ".../audit_report.json"
state.event_log.append("Data audit completed")
return state

def analysis_engineer_node(state: AgentState) -> AgentState:
    state.current_stage = "worker_run"
    # call Analysis Engineer wrapper or Snakemake workflow
    # save analysis_engineer_result.json
    state.analysis_engineer_result_path = ".../analysis_engineer_result.json"
    state.summary_csv_path = ".../summary.csv"
    state.event_log.append("Analysis Engineer run completed")
    return state

def fit_strategy_postdoc_node(state: AgentState) -> AgentState:
    state.current_stage = "fit_strategy"
    # read summary.csv and produce fit_strategy_report.json
    state.fit_strategy_report_path = ".../fit_strategy_report.json"
    state.event_log.append("Fit diagnostics completed")
    return state

```

注意: 本周的重点不是重新实现 Correlator Analyst 或 Analysis Engineer, 而是复用 Month 2 已经完成的模块。

3.6 Step 3: 定义 graph edge 逻辑

第一版 graph 可以是线性的:

```

START
-> correlator_analyst
-> worker
-> fit_strategy_postdoc
-> critical_referee
-> analysis_coordinator
-> save_state
-> END

```

但 analysis_coordinator 后应该预留条件分支:

```

analysis_coordinator
-> if low risk: memory_update
-> if medium/high risk: human_approval
-> if rerun_requested: targeted_rerun

```

Week 9 可以先实现线性版本, 把条件分支留到 Week 10。

3.7 Step 4: 保存 graph state

每次 graph run 结束时，都应保存：

```
outputs/graph_demo/<case_id>/graph_state.json
outputs/graph_demo/<case_id>/graph_run.log
```

graph_state.json 是本月最重要的文件之一。它应该回答：

- 系统运行到了哪一步？
- 每一步生成了哪些文件？
- 有没有 warning 或 error？
- Analysis Coordinator 判断的 risk level 是什么？
- 是否需要 human approval？
- 是否提出 targeted rerun？

3.8 Week 9 Codex Prompt

We are building Month 3 of an AI-first LQCD meson DA analysis-agent prototype. Month 1 implemented config-driven CLI and schema. Month 2 implemented Snakemake workflow, Analysis Engineer wrapper, Fit Strategy Postdoc, Critical Referee, and rule-based Analysis Coordinator.

Task:

Create the first LangGraph-style state-machine wrapper for the existing pipeline.

Files to create:

- src/lqcd_agent/graph/state.py
- src/lqcd_agent/graph/nodes.py
- src/lqcd_agent/graph/run_graph.py
- tests/test_graph_state.py
- tests/test_graph_smoke.py

Requirements:

1. Define an AgentState schema with case_id, case_config_path, output_dir, stage status, paths to generated outputs, risk level, human approval fields, rerun fields, warnings, errors, and event_log.
2. Implement graph nodes as small functions: correlator_analyst_node, analysis_engineer_node, fit_strategy_postdoc_node, critical_referee_node, analysis_coordinator_node, save_state_node.
3. For now, nodes may call existing modules if available; otherwise they should use clearly marked placeholder calls and check expected output paths.
4. Implement a CLI command:

```
python -m lqcd_agent.graph.run_graph --case configs/cases/demo.yaml --output-dir
outputs/graph_demo/demo
```

5. Save `graph_state.json` and `graph_run.log`.

6. Add `pytest` tests for `AgentState` validation and a smoke test using a tiny demo case.

Constraints:

- Do not rewrite existing Analysis Engineer/Analysis Coordinator modules.
- Do not perform broad shell operations.
- Do not overwrite existing outputs unless explicitly allowed.
- Keep the graph simple and readable.
- All generated files should be under the specified `output_dir`.

After implementation, explain how to run the graph and which tests were added.

3.9 Week 9 命令示例

```
# 运行 graph
python -m lqcd_agent.graph.run_graph \
  --case configs/cases/l64c64a076_demo.yaml \
  --output-dir outputs/graph_demo/l64c64a076
# 检查 graph state
cat outputs/graph_demo/l64c64a076/graph_state.json
# 运行 smoke test
pytest tests/test_graph_state.py tests/test_graph_smoke.py -q
```

3.10 Week 9 常见错误

- `node` 之间传递随意自由文本：必须用 `typed state` 字段，不能靠自然语言传递决策信息。
- 每个 `node` 重写已有模块：`node` 应该封装已有模块的调用，不是重新实现。
- `graph state` 不保存：每次 `run` 结束都应保存 `graph_state.json`。

3.11 Week 9 导师检查问题

- 每个 `node` 的输入输出是否结构化（是 `typed state` 字段，不是自由文本）？
- `graph state` 是否能被保存和恢复？
- 学生能否画出 `graph` 的流程图并解释每个 `node` 的作用？

3.12 Week 9 验收标准

Week 9 完成时，必须满足：

- 能运行 `python -m lqcd_agent.graph.run_graph ...`；

- 能生成 `graph_state.json`;
- `graph state` 中包含各模块输出路径;
- 每个 `node` 的输入输出是结构化 `state`, 而不是自由文本;
- 至少有两个测试: `state schema test` 和 `graph smoke test`;
- 学生能画出当前 `graph` 的流程图并解释每个 `node` 的作用。

4 Week 10: Risk-Based Human Approval Gate 与 Case Memory

4.1 本周目标

Week 10 的目标是实现科研可靠性中最关键的两个机制。本周最重要的设计决定是：**Human Approval Gate 是 risk-based conditional gate, 不是 always-on sequential gate。**

1. **Risk-Based Human Approval Gate**: 按风险等级决定人类介入深度。Low risk 自动推进, Medium risk 标记 pending candidate, High risk 生成 approval packet 并暂停, Critical risk 禁止 AI 推进。
2. **Case memory**: 系统按风险等级分层保存 accepted / rejected / pending cases。本周也保留 Targeted Rerun Request 作为 Analysis Coordinator 的关键输出。

4.2 学生需要理解的理念: Risk-Based Autonomy

本项目的长期目标是尽量减少人为介入, 让 AI agents 替代大部分 human labor 和 routine scientific decisions。一个先进的科研 AI agent 系统应该理解:

- **Low risk**: AI agents 自动执行、自动记录、自动推进 workflow, 不需要人类介入;
- **Medium risk**: AI agents 继续推进, 标记 human_review_recommended=true, 结果只能作为 pending candidate, 不能自动升级为 final accepted result;
- **High risk**: AI agents 生成 approval_packet/, workflow 在 promotion / final decision 前暂停, 等待人类审批;
- **Critical risk**: AI 不允许继续推进科学结论, 必须由 human PI / advisor 亲自决定。

人类角色不是 routine executor, 而是 **high-value decision maker** 和 **final accountability holder**。

4.3 Step 1: 定义 Risk-Based Autonomy 与 AI 决策边界

第一版通过结构化规则判断风险等级 (rule-based risk assessment)。

AI 可以自动决定 (无需人类介入):

- missing files 检测和报错
- config validation failure 诊断
- routine rerun (参数未变的重复执行)
- failed fit 自动标记
- bad candidate fit 自动拒绝 (基于明确统计阈值)

- diagnostic plot 生成和归档
- targeted rerun suggestion 提出
- low-risk workflow 自动推进和 memory update

AI 可以推荐但不能自动最终批准：

- central fit candidate 选择
- systematic variation set 定义
- data exclusion 建议（标记 exclusion candidate）
- accepted candidate promotion（标记 pending promotion）
- physics interpretation 初步撰写

必须 Human Approval Gate（不可绕过）：

- final central value 确定
- final systematic uncertainty 确定
- final accepted result promotion（pending → accepted）
- final data exclusion 确认
- paper-level claim 审批
- major analysis strategy change 批准
- overriding Critical Referee major objection

4.4 Step 2: 定义 Human Approval Packet 文件结构

当 Analysis Coordinator 判定为 High risk 时，系统生成完整的审批包目录，而非单一请求 JSON：

```
outputs/<case_id>/approval_packet/
  00_requested_decision.md      # 需要人类做哪个决定
  01_summary_for_human.md      # 人类可读的摘要（含关键数字和图表引用）
  02_key_plots.pdf              # 关键诊断图（effective mass、fit stability 等）
  03_fit_comparison.csv         # 候选 fit 对比表
  04_critical_referee_report.json # Critical Referee 完整报告
  05_ai_recommendation.json     # AI 推荐及理由
  06_decision_options.yaml      # 可选决策项及后果
  07_human_decision.yaml        # 人类填写后返回（系统读取）
```

06_decision_options.yaml 示例：

```

decision_request:
  case_id: "l64c64a076_m140"
  risk_level: "high"
  question: "Select central fit candidate"
  ai_recommendation:
    preferred: "2-state fit (tmin=5)"
    reason: "Better chi2 stability across scan window"
  options:
    - id: "option_a"
      label: "2-state fit (tmin=5) as central"
      risk_if_chosen: "medium"
      ai_preference: "recommended"
    - id: "option_b"
      label: "1-state fit (tmin=8) as central"
      risk_if_chosen: "medium"
      ai_preference: "alternative"
    - id: "option_c"
      label: "Request additional scan with tmin>=7"
      risk_if_chosen: "low"
      ai_preference: "safe_fallback"
  deadline: "2026-06-01"
  fallback_if_no_response: "hold_and_remind"

```

07_human_decision.yaml 由人类填写:

```

human_decision:
  case_id: "l64c64a076_m140"
  chosen_option: "option_a"
  approver: "advisor_name"
  comments: "Accepted 2-state as central. Add 1-state as systematic in next round."
  timestamp: "2026-05-20T15:00:00"

```

4.5 Step 3: 实现条件分支 Graph (替代线性流程)

Analysis Coordinator 之后根据 risk level 走不同分支:

```

analysis_coordinator_node(state) -> risk_level:
  if risk_level == "low":
    -> auto_continue_node
    -> write_decision_log
    -> memory_curator_update (save as accepted)
    -> END

  if risk_level == "medium":
    -> continue_as_pending_node

```

```

-> flag human_review_recommended = true
-> memory_curator_update (save as pending_candidate)
-> END

if risk_level == "high":
    -> generate_approval_packet_node
    -> human_approval_gate_node
        -> if approved: promote_to_accepted -> memory_curator_update -> END
        -> if request_rerun: generate_rerun_request -> back to analysis_engineer
        -> if rejected: save_as_rejected -> memory_curator_update -> END

if risk_level == "critical":
    -> HALT
    -> notify_human_pi
    -> await_human_decision

if status == "failed" or "inconsistent":
    -> generate_rerun_request -> back to analysis_engineer

```

Human approval node 的职责：

- 生成完整的 approval_packet/ 目录及所有 7 个文件；
- 如果没有 07_human_decision.yaml，则把 state 标记为 needs_human_approval；
- 读取 human decision 后正确路由（approved → promote / rejected → rejected memory / request_rerun → targeted rerun）；
- 超时未回复按 fallback_if_no_response 处理（默认 hold_and_remind）。

4.6 Step 4: 定义 Targeted Rerun Request（含风险上下文）

Targeted rerun 与 risk level 关联：high risk 的 targeted rerun 在人类审批前一般不应自动执行。

```

{
  "case_id": "l64c64a076_demo",
  "risk_level": "high",
  "trigger": "Critical Referee objection or Analysis Coordinator decision",
  "rerun_target": "fit_scan",
  "reason": "E0 shifts by 1.8 sigma between tmin=5 and tmin=7.",
  "suggested_changes": {
    "tmin_values": [5, 6, 7, 8, 9],
    "models": ["2state"],
    "increase_bootstrap": true
  },
  "expected_outputs": [

```

```

    "summary.csv",
    "fit_stability.pdf",
    "fit_strategy_report.json"
  ]
}
```

4.7 Step 5: 建立 Case Memory (按风险等级分层)

推荐先用 YAML 或 JSON。Memory 按 risk level 分层记录：

```

memory/
  accepted_cases.yaml      # low risk + auto accepted
  pending_cases.yaml      # medium risk + awaiting human review
  rejected_cases.yaml      # high/critical risk + human rejected 或 auto rejected
  human_decisions.yaml     # 所有人类决定的完整记录 (audit trail)
  failure_patterns.md      # 常见失败模式总结
```

Accepted case 示例 (必须在 memory 中记录 risk level 和 human approval 状态)：

```

- case_id: l64c64a076_demo
  ensemble: l64c64a076
  channel: pion
  status: accepted
  selected_fit: 2state_tmin5_tmax15
  risk_level: high
  human_approved: true
  approval_packet: outputs/.../approval_packet/
  human_decision: option_a
  key_reasons:
    - 2-state fit stable in preferred window
    - chi2_dof acceptable
    - critical referee review found no blocking issue
  output_paths:
    decision_log: outputs/.../decision_log.json
    summary_csv: outputs/.../summary.csv
  notes: Accepted with human approval. Production requires z-dependence check.
```

4.8 Week 10 命令示例

```

# 运行 graph with approval gate
python -m lqcd_agent.graph.run_graph \\\
  --case configs/cases/l64c64a076_demo.yaml \\\
  --output-dir outputs/graph_demo/l64c64a076
# 检查 approval packet
ls outputs/graph_demo/l64c64a076/approval_packet/
```

```
cat outputs/graph_demo/164c64a076/approval_packet/06_decision_options.yaml
# 模拟人类填写决策
cp templates/human_decision.yaml \
  outputs/graph_demo/164c64a076/approval_packet/07_human_decision.yaml
# 继续 graph run (读取 human decision)
python -m lqcd_agent.graph.resume \
  --output-dir outputs/graph_demo/164c64a076
# 查看 memory
cat memory/accepted_cases.yaml
cat memory/pending_cases.yaml
# 运行测试
pytest tests/test_risk_based_approval.py -q
```

4.9 Week 10 常见错误

- 把 **Human Approval Gate** 当成 **always-on gate**: 它只在 high/critical risk 触发。Low/medium risk 应自动推进。
- 只有单一 **JSON** 请求, 没有完整 **approval packet**: high risk 审批需要完整的 context (plots、referee report、decision options), 不能只给一个数字。
- 没有 **fallback** 机制: 如果人类超时未回复, 系统应能按 `fallback_if_no_response` 处理, 而不是无限等待。
- **Memory** 不记录 **risk level** 和 **human approval** 状态: 每个 case entry 都必须包含 `risk_level` 和 `human_approved` 字段。

4.10 Week 10 导师检查问题

- High risk case 是否生成了完整的 approval packet (7 个文件)?
- 没有 human decision 时, state 是否停留在“needs_human_approval”而不是静默推进?
- Medium risk case 是否标记为 pending candidate 而非 accepted?
- Critical risk case 是否 HALT 并通知 human PI?
- Memory 是否按 risk level 分层保存? 是否记录了 human decision?
- 学生能否解释 Human Approval Gate 为什么是 exception path 而不是 always-on gate?

4.11 Week 10 Codex Prompt

We are implementing the Risk-Based Human Approval Gate for the Month 3 LQCD meson DA analysis-agent prototype.

Task:

Implement conditional graph routing based on risk level, approval packet generation, and case memory.

Files to create or edit:

- src/lqcd_agent/graph/nodes.py
- src/lqcd_agent/graph/edges.py
- src/lqcd_agent/graph/approval_packet.py
- src/lqcd_agent/memory/schema.py
- src/lqcd_agent/memory/store.py
- tests/test_risk_based_approval.py
- tests/test_memory_update.py

Requirements:

1. After Analysis Coordinator, calculate risk_level based on rules:
 - Low: all analyses pass, fit stable, chi2 good, no referee flags
 - Medium: model dependence < 2 sigma, minor referee flags
 - High: major referee objection, fit selection needed, data exclusion proposed
 - Critical: unresolved physics contradiction, normalization failure
2. Low risk: auto continue + write decision_log + update accepted memory.
3. Medium risk: continue as pending candidate + flag human_review_recommended.
4. High risk: generate approval_packet/ directory with all 7 files.
5. Critical risk: HALT graph, notify human PI.
6. Parse 07_human_decision.yaml from approval_packet/.
7. Add tests for each risk level routing (low/medium/high/critical).

Constraints:

- Do not require a web UI.
- Do not silently promote high-risk results.
- Keep all memory files human-readable YAML.
- Human Approval Gate is an exception path, not the default path.

4.12 Week 10 验收标准

Week 10 完成时，必须满足：

- Low risk case 自动推进并进入 accepted_cases.yaml;
- Medium risk case 标记 pending candidate, human_review_recommended=true;
- High risk case 生成完整 approval_packet/(7 个文件), graph 暂停在 needs_human_approval;
- 人类 decision 被正确解析 (approved → promote / rejected → rejected memory);
- Critical risk case 触发 HALT, 通知 human PI;
- Memory 按 risk level 分层保存, 每个 entry 含 risk_level 和 human_approved 字段;

- 至少有 4 个测试覆盖 low/medium/high/critical 四种 risk level 的路由；
- 学生能解释 Human Approval Gate 为什么是 risk-based conditional gate 而不是 always-on sequential gate。

5 Week 11: Evaluation Harness、Failure Cases 与自动报告生成

5.1 本周目标

Week 11 的目标是让系统不仅能跑一个 case，而且能被测试和评估。一个科研 agent 系统如果没有 evaluation harness，就很难知道它的判断是否稳定、是否可回归、是否被新代码破坏。本周要构建三类固定 cases：

1. **Good case:** plateau 稳定，fit 质量好，Analysis Coordinator 应该给 low risk 或 approve recommendation。
2. **Ambiguous case:** 1-state / 2-state 有差异，或者 tmin dependence 明显，Analysis Coordinator 应该要求 human approval。
3. **Bad case:** 无稳定 plateau 或所有 fit 都失败，Analysis Coordinator 应该 reject 或 request rerun。

5.2 Step 1: 建立 evaluation cases 目录

推荐目录：

```
evaluation/
cases/
  good_plateau.yaml
  ambiguous_excited_state.yaml
  bad_no_plateau.yaml
expected/
  good_plateau_expected.json
  ambiguous_excited_state_expected.json
  bad_no_plateau_expected.json
run_evaluation.py
evaluation_report.md
```

5.3 Step 2: 定义 expected behavior

不要只测试代码是否能跑，还要测试 Analysis Coordinator 的行为是否合理。

Good case expected:

```
{
  "case_id": "good_plateau",
  "expected_risk_levels": ["low"],
```

```

"expected_human_approval_required": false,
"expected_decision_contains": ["stable", "acceptable chi2"]
}

```

Ambiguous case expected:

```

{
  "case_id": "ambiguous_excited_state",
  "expected_risk_levels": ["medium", "high"],
  "expected_human_approval_required": true,
  "expected_decision_contains": ["model dependence", "tmin"]
}

```

Bad case expected:

```

{
  "case_id": "bad_no_plateau",
  "expected_risk_levels": ["high"],
  "expected_human_approval_required": true,
  "expected_decision_contains": ["no stable", "rerun"]
}

```

5.4 Step 3: 实现 run_evaluation.py

run_evaluation.py 应该循环运行 evaluation cases:

```

python evaluation/run_evaluation.py --cases evaluation/cases --output-dir outputs/
evaluation

```

输出:

```

outputs/evaluation/
good_plateau/
  graph_state.json
  decision_log.json
ambiguous_excited_state/
  graph_state.json
  decision_log.json
bad_no_plateau/
  graph_state.json
  decision_log.json
evaluation_summary.csv
evaluation_report.md

```

5.5 Step 4: 自动生成 final analysis report

除了 agent 内部的 JSON 文件，还需要一个人能读懂的报告。第一版可以生成 Markdown。报告应包括：

- case metadata;
- data audit summary;
- Analysis Engineer outputs;
- fit diagnostics summary;
- Critical Referee review;
- Analysis Coordinator recommendation;
- risk level;
- human approval status;
- rerun recommendation;
- links to plots and JSON outputs;
- final notes.

推荐文件:

```
src/lqcd_agent/report/  
__init__.py  
generate_report.py  
templates.py
```

报告示例结构:

```
# Analysis Agent Report: 164c64a076_demo  
  
## 1. Case Metadata  
...  
  
## 2. Data Audit  
...  
  
## 3. Analysis Engineer Execution  
...  
  
## 4. Fit Diagnostics  
...  
  
## 5. Critical Referee Review  
...  
  
## 6. Analysis Coordinator Decision  
...
```

```
## 7. Human Approval
...

## 8. Recommended Next Action
...

## 9. Reproducibility
Command:
...
Config:
...
Git commit:
...
```

5.6 Step 5: 加入 git commit hash 和 environment info

为了提高可追责性，报告中最好记录：

- git commit hash;
- Python version;
- package versions;
- run timestamp;
- config path;
- output path。

第一版可以简单实现：

```
import subprocess

def get_git_commit():
    try:
        return subprocess.check_output(["git", "rev-parse", "HEAD"], text=True).strip()
    except Exception:
        return "unknown"
```

注意：只允许这种可控的小命令，不允许 broad shell operations。

5.7 Week 11 Codex Prompt

We are building an evaluation harness and report generator for the Month 3 AI-first LQCD meson DA analysis-agent prototype.

Task:

Create fixed evaluation cases and a script that runs the graph on each case, compares the Analysis Coordinator decision with expected behavior, and generates an evaluation report.

Files to create:

- evaluation/cases/good_plateau.yaml
- evaluation/cases/ambiguous_excited_state.yaml
- evaluation/cases/bad_no_plateau.yaml
- evaluation/expected/*.json
- evaluation/run_evaluation.py
- src/lqcd_agent/report/generate_report.py
- tests/test_evaluation_expected_behavior.py

Requirements:

1. The evaluation script should run the graph for each case.
2. It should collect risk_level, human_approval_required, rerun_requested, selected_fit, and decision reasons.
3. It should compare these fields with expected behavior.
4. It should write evaluation_summary.csv and evaluation_report.md.
5. The report generator should produce a human-readable Markdown report for one case.
6. Include git commit hash and run timestamp if available.
7. Add tests for expected behavior comparison.

Constraints:

- Do not depend on real large data.
- Use small fake or golden-example-derived cases.
- Keep the evaluation cases deterministic.
- Do not hide mismatches; report them clearly.

5.8 Week 11 命令示例

```
# 运行 evaluation
python evaluation/run_evaluation.py \
  --cases evaluation/cases \
  --output-dir outputs/evaluation
# 查看 evaluation 结果
cat outputs/evaluation/evaluation_summary.csv
cat outputs/evaluation/evaluation_report.md
# 生成 single case report
python -m lqcd_agent.report.generate_report \
```

```
--case outputs/l64c64a076_demo \  
--output outputs/l64c64a076_demo/final_report.md  
# 运行测试  
pytest tests/test_evaluation_expected_behavior.py -q
```

5.9 Week 11 常见错误

- **Evaluation cases** 依赖真实数据：使用 fake/synthetic 数据，保证 evaluation 在任何环境可运行。
- **Expected behavior** 不具体：不能只写”pass”，必须写 expected risk level 和 expected human approval status。
- **Report** 生成全是 placeholder：每个 section 必须有实质性内容。

5.10 Week 11 导师检查问题

- Evaluation 是否覆盖 good/ambiguous/bad 三类 case?
- Analysis Coordinator 行为是否与 expected behavior 一致? mismatch 是否被清楚报告?
- Final report 是否包含全部 10 个 section? 每个 section 是否有实质性内容?

5.11 Week 11 验收标准

Week 11 完成时，必须满足：

- 至少有 good、ambiguous、bad 三类 evaluation cases;
- 可以一键运行 evaluation;
- 生成 evaluation_summary.csv;
- 生成 evaluation_report.md;
- 对 Analysis Coordinator 的 risk level 和 human approval 行为有测试;
- 报告中包含复现命令、config、输出路径和 git commit;
- 学生能解释 evaluation harness 为什么比 “跑通一个 demo” 更重要。

6 Week 12: 最终 Demo、文档整理与产品化设计

6.1 本周目标

Week 12 的目标是把前三周的模块整理成一个导师和组内成员都能理解、能运行、能评价的完整 demo。

最终 demo 不要求系统已经完美处理所有真实数据，但必须展示：

- 真实科研 workflow 的雏形；
- agent roles 的清晰分工；
- structured state 和 structured outputs；
- reviewer / skeptic 机制；
- Analysis Coordinator decision log；
- human approval gate；
- case memory；
- evaluation harness；
- 最终报告。

6.2 Step 1: 整理一键 demo 命令

建议提供一个 Makefile 或 simple shell script，但脚本要简单透明。

```
make demo
```

或者：

```
bash scripts/run_month3_demo.sh
```

脚本内容可以包括：

```
#!/usr/bin/env bash
set -euo pipefail

python -m lqcd_agent.graph.run_graph \
  --case configs/cases/l64c64a076_demo.yaml \
  --output-dir outputs/month3_demo/l64c64a076_demo

python evaluation/run_evaluation.py \
  --cases evaluation/cases \
  --output-dir outputs/evaluation
```

6.3 Step 2: 整理最终输出目录

最终 demo 输出目录建议如下：

```
outputs/month3_demo/164c64a076_demo/  
  graph_state.json  
  audit_report.json  
  analysis_engineer_result.json  
  summary.csv  
  fit_strategy_report.json  
  critical_referee_report.json  
  decision_log.json  
  approval_packet/  
  approval_packet/07_human_decision.yaml      # if provided  
  rerun_request.json                          # if needed  
  final_report.md  
  plots/  
    effective_mass.pdf  
    fit_stability.pdf  
  logs/  
    graph_run.log  
    run.log
```

6.4 Step 3: 补全 README

主 README 至少应包括：

1. 项目目标；
2. 架构图；
3. Analysis Engineer / Analysis Coordinator / Critical Referee / Analysis Memory Curator 的职责；
4. 安装方法；
5. 如何运行 Month 1 CLI；
6. 如何运行 Month 2 Snakemake workflow；
7. 如何运行 Month 3 graph demo；
8. 如何运行 evaluation；
9. 输出文件解释；
10. 当前限制；
11. 下一步 roadmap。

6.5 Step 4: 画 architecture diagram

可以先用 Mermaid 写在 README 中：

```

flowchart TD
    A[Case Config] --> B[Correlator Analyst]
    B --> C[Analysis Engineer Agent]
    C --> D[Snakemake Workflow]
    D --> E[Fit Strategy Postdoc]
    E --> F[Matrix Element Analyst]
    F --> G[Critical Referee]
    G --> H[Analysis Coordinator]
    H -->|low risk| I[Auto Continue + Memory Update]
    H -->|medium risk| J[Continue as Pending + Human Review Recommended]
    H -->|high risk| K[Generate Approval Packet + Human Approval Gate]
    H -->|critical risk| L[HALT + Notify Human PI]
    H -->|failed| M[Targeted Rerun Request]
    M --> C
    K -->|approved| N[Promote to Accepted + Memory Update]
    K -->|rejected| O[Rejected Memory]
    K -->|request rerun| M
    I --> P[Final Report]
    J --> P
    N --> P
  
```

6.6 Step 5: 准备最终汇报

学生最终汇报应包括 8–10 页 slides 或一份短报告：

1. 项目问题：为什么 LQCD meson DA analysis 需要 AI-first agent?
2. 系统架构：Analysis Engineer、Critical Referee、Analysis Coordinator、Human Approval Gate、Analysis Memory Curator。
3. Month 1：CLI、schema、pytest。
4. Month 2：Snakemake、Analysis Engineer、Fit Strategy Postdoc、Critical Referee、decision log。
5. Month 3：LangGraph state machine、human approval、targeted rerun、case memory、evaluation。
6. Demo case 输出。
7. 一个 good case 和一个 bad/ambiguous case 的对比。
8. 当前系统限制。

9. 未来升级方向。

10. 学生自己的收获：学会了哪些 AI-first research workflow 技能。

6.7 Week 12 Codex Prompt

We are preparing the final Month 3 demo for an AI-first LQCD meson DA analysis-agent prototype.

Task:

Create final demo scripts, improve README documentation, and add a Mermaid architecture diagram.

Files to create or edit:

- README.md
- scripts/run_month3_demo.sh
- docs/architecture.md
- docs/month3_final_demo.md
- Makefile or equivalent simple command runner

Requirements:

1. Provide one clear command to run the Month 3 demo.
2. Provide one clear command to run evaluation cases.
3. Document the roles: Correlator Analyst, Analysis Engineer, Fit Strategy Postdoc, Critical Referee, Analysis Coordinator, Human Approval Gate, Analysis Memory Curator.
4. Add a Mermaid architecture diagram.
5. Document all important output files and what they mean.
6. Add a limitations section and future roadmap.
7. Do not hide known limitations.

Constraints:

- Do not add a complex UI.
- Do not introduce new heavy dependencies.
- Keep documentation clear enough for a new graduate student to follow.

6.8 Week 12 命令示例

```
# 一键 demo
bash scripts/run_month3_demo.sh
# 或
make demo
# 查看最终输出
ls -R outputs/month3_demo/l64c64a076_demo/
# 查看 README
```

```
cat README.md
# 查看 architecture diagram
open docs/architecture.pdf
```

6.9 Week 12 常见错误

- 只有代码没有文档：README 必须让另一个学生能照着复现。
- Demo 脚本依赖特定路径：用相对路径和 config 驱动，不 hard-code 绝对路径。
- Architecture diagram 太复杂或太简单：应能在一页内展示核心 agent flow。

6.10 Week 12 导师检查问题

- 是否能一键运行 demo (bash scripts/run_month3_demo.sh 或 make demo)?
- README 是否足够另一个学生跟着复现?
- Architecture diagram 是否清楚展示 agent 角色和 flow?
- Final demo 是否展示了完整的 9 项能力 (参考主 guide Ch 12)?

6.11 Week 12 验收标准

Week 12 完成时，必须满足：

- 有一键 demo 命令；
- 有一键 evaluation 命令；
- README 能让新学生理解并运行项目；
- architecture diagram 清楚；
- final report 能自动生成；
- 至少展示一个 good case 和一个 ambiguous/bad case；
- case memory 有 accepted/rejected/suspicious 示例；
- 学生能现场解释每个 agent 的输入、输出和职责边界。

7 Month 3 最终交付清单

第三个月结束时，学生应该提交：

1. 完整 Git repo;
2. Month 3 demo command (一键运行);
3. LangGraph 或等价 state-machine 代码;
4. `graph_state.json` 示例;
5. `approval_packet/` 完整示例 (7 个文件 + human decision);
6. `rerun_request.json` 示例;
7. `memory/accepted_cases.yaml`、`rejected_cases.yaml`、`pending_cases.yaml`、`human_decisions.yaml`;
8. evaluation cases (good/ambiguous/bad);
9. `evaluation_summary.csv` 和 `evaluation_report.md`;
10. `final_report.md` (含 10 个 section);
11. `demo_script.md` (或 Makefile);
12. `architecture_diagram.pdf`;
13. README 和 architecture document;
14. 一份 8–10 页最终汇报 slides。

8 导师验收 checklist

导师可以按以下 checklist 检查 Month 3 成果：

8.1 Agent architecture

- 是否有明确 agent roles?
- 每个 agent 是否有清楚输入输出?
- 是否有统一 state?
- 是否避免了自由文本乱传?
- 是否支持失败状态?

8.2 Risk-Based Human Approval Gate

- 高风险结果是否会请求人工审批?
- 没有审批时是否不会自动进入 accepted memory?
- approval response 是否被记录?
- rejected case 是否被保存?

8.3 Targeted rerun

- Analysis Coordinator 是否能指出重跑目标?
- rerun reason 是否明确?
- suggested changes 是否结构化?
- 是否避免无意义全量重跑?

8.4 Memory

- 是否有 accepted/rejected/suspicious memory?
- memory 是否人类可读?
- memory 是否记录理由, 而不只是记录结果?
- 是否能用于未来 case comparison?

8.5 Evaluation

- 是否有 good/ambiguous/bad 固定 cases?
- Analysis Coordinator 行为是否被测试?
- 新代码是否能通过 regression tests?
- evaluation report 是否清楚?

8.6 科研可靠性

- 是否记录 config、git commit、运行命令和输出路径?
- 是否有 decision log?
- 是否有 critical referee review?
- 是否区分了 demo result 和 production physics result?
- 是否清楚说明系统限制?

9 升华：理想产品应该呈现出的效果

9.1 理想中的产品不是“自动跑脚本”，而是“科研分析驾驶舱”

本项目的最终目标不应该停留在“AI 帮我跑几个脚本”或“Codex 帮我写代码”。理想产品应该逐步发展成一个 **LQCD meson DA analysis cockpit**，也就是一个科研分析驾驶舱。

在这个理想系统中，研究者给出一个物理目标，例如：

在 ensemble A 上分析某个 meson DA matrix element，检查 2pt fit、ratio fit、normalization、z-dependence、momentum dependence，并判断是否可以进入下一步 matching 或 continuum analysis。

系统应该能够：

1. 自动识别所需数据是否齐全；
2. 自动生成或检查 config；
3. 自动运行可复现 workflow；
4. 自动汇总 fit candidates；
5. 自动发现异常趋势；
6. 自动给出少量高价值 rerun 建议；
7. 自动写出 decision log；
8. 自动生成带图、带路径、带理由的 analysis report；
9. 对高风险判断请求人工审批；
10. 把人工批准或拒绝的 case 写入 memory；
11. 在未来遇到类似 case 时参考历史经验。

这才是科研 AI agent 的核心价值：它不是替代科研者，而是把科研者的分析流程结构化、审计化、可复现化，并逐步学习科研判断。

9.2 产品最终应该让导师看到什么

一个成熟的 demo 不应该只展示代码，而应该展示一个完整闭环：

- 输入：一个 case config 和数据路径；
- 执行：Snakemake 管理计算依赖，Analysis Engineer 安全运行；
- 分析：Fit Strategy Postdoc 产生 fit diagnostics；

- 批判: Critical Referee 找出潜在问题;
- 决策: Analysis Coordinator 给出 recommendation、risk level 和 reasons;
- 审批: Human Approval Gate 对 high/critical risk 结果把关 (exception path, 非 always-on gate);
- 记忆: Analysis Memory Curator 保存 accepted/rejected/suspicious cases;
- 报告: 系统自动生成 final report;
- 评估: evaluation harness 证明系统在 good/ambiguous/bad cases 上行为合理。

如果这九个环节都能跑通, 即使每个环节还很初级, 这个项目也已经具有很高价值。

9.3 未来升级方向: 如何变成更高级的系统

9.3.1 升级 1: 从 rule-based Analysis Coordinator 到 LLM-assisted Analysis Coordinator

第一版 Analysis Coordinator 使用规则是正确的, 因为规则透明、可测试、可控。下一阶段可以引入 LLM, 但不要让 LLM 直接决定结果。更好的方式是:

- LLM 读取 structured diagnostics;
- LLM 生成 human-readable reasoning;
- LLM 提出 candidate explanations;
- LLM 提出 targeted rerun plan;
- rule-based guardrail 检查 LLM 输出是否符合 schema 和安全规则;
- 中高风险结论仍需 human approval。

也就是说, LLM 应该先做 **reasoning assistant**, 再逐步参与 **strategy recommendation**, 最后才可能在积累足够 case memory 后承担更高层次的判断。

9.3.2 升级 2: 从 YAML memory 到 SQLite / DuckDB analysis memory

当 case 数量增多后, YAML 会不方便检索。可以升级到 SQLite 或 DuckDB:

- 保存 ensemble、channel、momentum、z、fit model、fit window、risk level;
- 支持查询类似 cases;
- 支持统计常见失败原因;
- 支持生成 performance dashboard;
- 支持从 rejected cases 中总结规则。

这会让系统从“能记录经验”升级为“能检索和利用经验”。

9.3.3 升级 3: 加入 richer observability

高级 agent 系统需要 observability, 也就是可观察性。未来可以加入:

- MLflow 或类似工具记录 run metadata;
- 每个 node 的耗时、输入、输出、失败率;
- Codex/LLM 调用成本和 latency;
- 每次 rerun 的原因;
- 每个 case 的审批历史;
- dashboard 展示 pipeline health。

目标不是为了好看, 而是为了回答: 系统什么时候可靠, 什么时候不可靠, 哪里最容易失败。

9.3.4 升级 4: 从单 case 到 ensemble-level campaign

当前 demo 可能只处理一个 golden example。下一阶段应扩展到多个 ensembles、多个 momenta、多个 z values:

- 自动生成 campaign config;
- Snakemake 批量调度;
- Analysis Coordinator 对每个 case 生成 decision;
- campaign-level Critical Referee 检查 ensemble dependence、 P_z dependence、 z -dependence;
- 生成 campaign summary report。

这是从“单点分析 agent”升级为“项目级分析 agent”的关键一步。

9.3.5 升级 5: 引入 multi-agent debate, 但必须有风险控制

借鉴先进 multi-agent 系统, 可以加入多个 reviewer:

- Fit Strategy Postdoc A (proponent): 提出可接受的 fit window 和候选方案;
- Fit Strategy Postdoc B (alternative): 提出不同的 fit 策略作为备选方案 (例如 1-state 替代 2-state);
- Critical Referee: 寻找不稳定点、模型依赖、方法学漏洞;
- Lattice QCD Physics Expert: 检查整体 LQCD 物理趋势是否合理;

- Perturbative Matching Expert: 检查 matching / factorization / renormalization 相关的系统误差;
- Analysis Coordinator: 综合各方意见。

但 debate 不能变成自由聊天。每个 reviewer 必须输出结构化 JSON:

```
{
  "reviewer_role": "CriticalReferee",
  "blocking_issues": [...],
  "non_blocking_warnings": [...],
  "recommended_action": "human_review",
  "confidence": "medium",
  "evidence_files": [...]
}
```

这样 debate 才能被测试、记录和复现。

9.3.6 升级 6: 加入真实 LQCD physics checks

未来 Analysis Coordinator 的物理检查应逐步增强, 包括:

- local matrix element consistency;
- $z=0$ normalization;
- real/imaginary symmetry;
- positive/negative z relation;
- P_z dependence;
- excited-state contamination;
- source-sink separation dependence;
- ensemble dependence;
- lattice-spacing dependence;
- comparison with known limits or previous accepted cases。

这些检查应该逐步从人工经验转化为 machine-checkable diagnostics。

9.4 如何变成 “the best”

要把这个项目做成最好的科研 AI agent 系统, 关键不是一开始接入最多模型或最多框架, 而是坚持以下原则:

1. **Every result is reproducible.** 任何结果都能追溯到 config、code version、input data 和 command。
2. **Every decision is logged.** 每个 fit recommendation 都必须有理由、有证据、有风险等级。
3. **Every high-risk decision has human approval.** AI 可以建议，但不能无审查地提升高风险结果。
4. **Every failure becomes memory.** 失败不是垃圾，而是训练未来 Analysis Coordinator 的案例。
5. **Every module has tests.** Codex 写的代码必须被 pytest 和 evaluation cases 固定下来。
6. **Every agent has a narrow responsibility.** Analysis Engineer 不做最终物理判断，Critical Referee 不直接修改结果，Analysis Coordinator 不静默覆盖人工决定。
7. **Every upgrade is incremental.** 先 rule-based，再 LLM-assisted；先 single case，再 campaign；先 YAML memory，再 database；先 CLI，再 dashboard。
8. **Every physics check becomes structured.** 把导师脑中的判断逐步变成 diagnostics、rules、reports 和 memory。

最终最好的系统应该达到这样的状态：

导师提出物理目标，系统自动组织分析流程、运行计算、检查质量、提出少量高价值判断和 rerun 建议，并把所有代码、数据、图、日志、决策和人工审批保存下来。学生通过使用和扩展这个系统，不仅完成分析，也学习如何构建一个尽量自动化、只在关键点需要人类介入的现代科研系统。

这就是本项目的核心愿景：用 AI 尽可能承担科研工程里的重复劳动和常规分析，用工程化和审计机制守住科研可靠性，并让人类把精力集中在关键判断和高风险决策上。

10 给学生的最终提醒

第三个月会比前两个月更难，因为你不再只是写单个函数，而是在搭建一个小型科研系统。你需要始终记住：

- 不要只追求“能跑”，要追求“能解释为什么这样跑”；
- 不要只生成结果，要生成 result、log、diagnostics、decision 和 report；
- 不要害怕失败 case，失败 case 是训练 Analysis Coordinator 的材料；
- 不要盲目相信 Codex，必须让它写 tests，并自己运行和检查；
- 不要把 agent 设计成一个万能黑箱，要把它拆成可检查的小角色；

- 不要让 AI 静默决定高风险物理结论；high risk 必须触发 Human Approval Gate, critical risk 必须由 PI 亲自裁决。

如果你完成了 Month 3, 你已经不只是会用 AI 写代码, 而是初步掌握了如何构建一个 **AI-first scientific analysis agent system**。这是一种非常重要的现代科研能力。